UQAC
Université du Québec
à Chicoutimi

Université du Québec à Chicoutimi

Atelier II en intelligence artificielle

Projet 3 : Système de recommandation de livres

Professeur : Kevin Bouchard

Membres de l'équipe : Charlotte Chanudet, Mahaut Galice, Marie Howet

Tables des Matières

Introduction 2
Préparation des données 3
Dataset GNN 3
Dataset système de recommandation par similarité 5
Dataset commun 6
Approche 1 : mise en place des modèles sur deux datasets différents 6
Approche par similarité 7
Approche par GNN 8
Séparation train/test au niveau des arêtes 8
Construction du graphe biparti utilisateur-livre 8
Modèle LightGCN 9
Entraînement 9
Évaluation 10
Génération de recommandations 11
Approche 2 : Méthode hybride 12
Mise en place 12
Analyse des résultats 12
Conclusion 14
Sources : 14

Tables des Illustrations

Figure 1: Entête du dataset commun 4
Figure 2 - Distribution des notes sur le jeu de données 6
Figure 3 : avant (à gauche) et après (à droite) pré entrainement 6
Figure 4 : 2 exemples de recommandations par similarité 8
Figure 5 : Evaluation du modèle LightGCN 11

Figure 6 : Livres lus dans le train set (connus du modèle) 12
Figure 7 : Livres lus dans le test set (inconnus au modèle) 12
Figure 8 : Livres recommandés..... 12
Figure 9 : Top 20 des livres les plus populaires 13
Figure 10 : Répartition du dataset 14
Figure 11 : Résultat de la version BERT + KNN 14
Figure 12 : Résultat de la version hybride 14

Introduction

Ce projet porte sur la conception et l'étude de deux systèmes de recommandation de livres utilisant des approches techniques différentes l'un par approche par similarité l'autre par approche par GNN

L'objectif de ce rapport est de présentée nos différents choix d'implémentation, hases d'expérimentation et l'analyse des résultats obtenus. Dans un premier temps, chaque méthode a été développée et testée de manière indépendante sur des jeux de données (*datasets*) distincts. Dans un second temps, nous avons appliqué les deux approches sur un même jeu de données afin d'établir une comparaison directe et d'analyser l'efficacité de chaque algorithme dans un environnement identique.

Le lien pour le GitHub du projet : https://github.com/corailllum/8INF974_Projet3

Préparation des données

Avant d'expliquer les différentes démarches que nous avons mise en place, nous souhaitions revenir sur la préparation des différents jeux de données que nous avons mis en place. Tous les jeux de données que nous avons utilisés viennes du site Kaggle. Nous avons souhaité garder les données au maximum nous n'avons donc pas trop altérer les jeux de données afin de gardée un maximum de lignes pour les entrainements de model.

Figure 1: Entête du dataset commun

Dataset GNN

Le premier dataset que nous allons vous présenter est celui que nous avons utilisée pour le premier GNN mis en place : *Book Crossing Dataset*.

L'ensemble de données Book-Crossing comprend 3 jeux de données.

- *BX-Users* : il contient les utilisateurs avec leurs identifiants, la localisation et leur âge.
- *BX-Books* : il reprend les livres. Ils sont identifiés par leurs ISBN. De plus, certaines informations relatives au contenu sont fournies (Book-Title, Book-Author, Year-Of-Publication, Publisher). Des URL renvoyant vers des images de couverture sont également fournies.
- *BX-Book-Ratings* : il contient les informations relatives aux notes attribuées aux livres (ISBN) par les utilisateurs (User_ID). Les notes (Book-Rating) sont soit explicites, exprimées sur une échelle de 1 à 10 (les valeurs les plus élevées indiquant une appréciation plus forte), soit implicites, exprimées par 0.

Nous avons appliqué les transformations suivantes sur ces données (cfr. *gnn_for_book_recommender.ipynb*):

- Suppression des valeurs manquantes
- Suppression des notations implicites (Book-Rating = 0)
- Suppression des ISBN présents dans *BX-Book-Ratings* mais pas *BX-Books*

Plusieurs filtrages ont été effectués sur le jeu de donnés *BX-Book-Ratings*.

Un premier filtrage des données a été réalisé en fonction du nombre minimal d'interactions entre les utilisateurs et les livres. Pour être conservé dans le jeu de données, un utilisateur devait avoir noté au moins 10 livres, et un livre devait avoir reçu au moins 10 notes. Ce prétraitement permet d'éliminer les profils peu actifs ainsi que les ouvrages peu évalués afin d'obtenir un graphe plus robuste et représentatif.

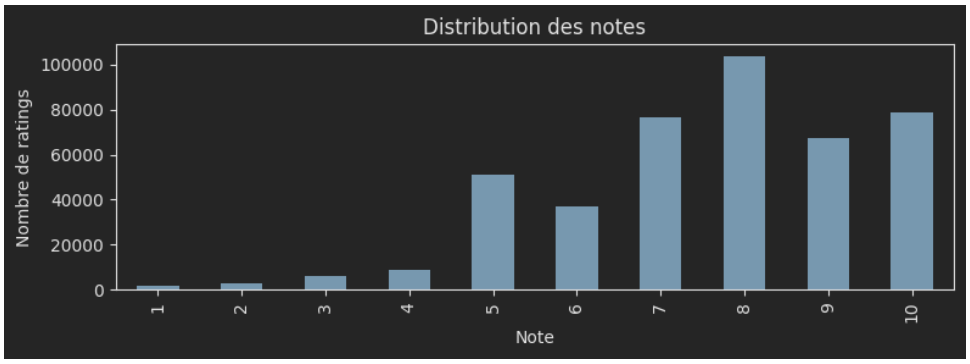

Figure 2 - Distribution des notes sur le jeu de données

Un second filtrage a ensuite été appliqué sur la note minimale considérée comme « positive » pour conserver une interaction utilisateur–livre et en faire une arête dans notre GNN. Comme le montre la distribution des notes (voir figure 1), la majorité des évaluations se situent au-delà de 5/10. Donc, 5 a été défini comme seuil minimal pour considérer une note comme favorable.

```
print(f"Ratings bruts  : {len(ratings):,}")
print(f"Livres uniques : {books['ISBN'].nunique():,}")
print(f"Users uniques  : {ratings['User-ID'].nunique():,}")
✓ [44] 3s 128ms

Ratings bruts  : 1,149,780
Livres uniques : 271,360
Users uniques  : 105,283
```

```
# Garder seulement les bonnes notes (>= 5)
good_ratings = ratings[ratings['Book-Rating'] >= 5].copy()

print(f"Nombre d'utilisateurs : {good_ratings['User-ID'].nunique():,}")
print(f"Nombre de livres      : {good_ratings['ISBN'].nunique():,}")
print(f"Nombre de ratings     : {len(good_ratings):,}")
✓ [52] 15ms

Nombre d'utilisateurs : 6,096
Nombre de livres      : 3,276
Nombre de ratings     : 69,572
```

Figure 3 : avant (à gauche) et après (à droite) pré entrainement

Finalement, après l'ensemble des étapes de prétraitement, le jeu de données a été considérablement réduit. Nous sommes passés de plus d'un million de notes à environ 70 000 interactions filtrées, de 105 000 utilisateurs à environ 6 000 utilisateurs actifs, et de 271 000 livres à environ 3 300.

Dataset système de recommandation par similarité

Pour ce système de recommandation, il nous faillait un data set qui possédait les descriptions des livres afin que l'on puisse les encodées. Nous avons donc choisi le dataset *GoodReads 100k book*s composé du CSV GoodReads_100k_books.csv. Nous avons par la suite appliqué les transformations suivantes sur les données (fichier test_reco_simi.py):

- Suppression des espace et mise en minuscule de tous les textes
- Suppression des lignes avec des titre ou descriptions manquante
- Filtrages des descriptions de moins de 50 caractères

- Suppression des titre doublons
- Séparation des données en groupe de train et de test
- Encodage des description (Sentence-bert)

Dataset commun

La dernière approche mise en place correspond à un “Mixte” des deux approches que nous avons mis en place au début. Nous avons donc dû trouver un dernier dataset qui possédait à la fois les rating utilisateur ainsi que les description complète des livres. Nous avons donc choisi le dataset Amazon Books Reviews composé des csv suivant Books_rating.csv et books_data.csv Les transformation faite sont les suivante (dans les fichiers preprocessing.py et pipeline_hybrid.py):

- Renommages des colonnes
- Jointure des datasets sur la colonne TITRE
- Suppression des valeurs manquante
- Filtrages des descriptions trop courte
- Filtrage des caractères ascii
- Filtrage des nombre minimaux d’interaction utilisateur-livre
- Filtrage sur la note minimale considérée comme positive.
- Encodage des identifiants
- Séparation des données en groupe de train et de test

Approche 1 : mise en place des modèles sur deux datasets différents

Notre première approche du projet a été de tester deux modèles d'implémentation diffèrents. Un par la similarité des descriptions de livre, grâce à un modèle kNN. Notre seconde approche portait sur une recommandation grâce au note utilisateur sur divers livres avec un model GNN. Lors de nos recherches, nous n'avions pas trouver de dataset unique compatible avec les deux model, c'est pour cela que nous étions parties sur deux dataset diffèrents.

Approche par similarité

Pour cette première approche, le but était de recommandée les livre en fonction de leur description. Pour se faire, la première étape a été d'encodée ces description en vecteur. Pour se faire nous avons utilisée l'embending Sentence-BERT (S-BERT). Ensuite, pour un livre donné, on calcule la similarité cosinus de son embending avec ceux des autres livres du corpus. Enfin on utilise un algorithme kNN pour trouver les livre les plus proches les uns des autres. Pour vérifier les résultats rapidement nous sommes passés par un test où l'on piochait un livre et on ressortait les 5 livres que l'algorithme aurait recommandé à une personne aillant aimer ce livre.

Cette approche est assez simple, et donc rapide à implémenter, en plus d'avoir l'avantage d'être rapide également à entraîner. Le souci principal est qu'il faut un dataset conséquent pour pouvoir avoir une grande base de résumés dans lesquels chercher des similarités. Le second point faible est que dès que l'on sort de l'alphabet latin on se retrouve avec des similarités très fortes car le code n'était pas capable de les prendre en compte, ce qui crée des similarités très fortes mais pas nécéssairement pertinentes.

Finalement cette approche nous donne quand même des résultats qui peuvent être intéressants, avec des similarités dans les 65% dans la majorité du temps, mais n'est pas idéale ni fiable sur le long terme. Surtout que l'on se rend compte par exemple sur la figure 4 que des livres semblant très proches ne sont pas forcément calculés comme étant très similaires, notamment 3 livres sur le CSS/HTML qui ne sont similaires qu'à 65% alors qu'ils traitent de sujets très proches.

```
   Titre       : CSS Mastery: Advanced Web Standards Solutions
   Auteur      : ['Cameron Moll', 'Andy Budd', 'Simon Collison']
   Categories  : ['Computers']
   Description : One of very few CSS books that focuses on teaching advanced-level CSS techniques CSS is a huge market—every web designer needs to know CSS to succeed in
 the modern web design marketplace Part of the "Solutions" series, and a follow-up from the best-selling Dan Cederholm book, Web Standards Solutions...

TOP 5 LIVRES SIMILAIRES
----------------------------------------------------------------------

  #1  Spring Into HTML and CSS
       Auteur      : ['Molly E. Holzschlag']
       Categories  : ['Computers']
       Similarite  : 65.8%
       Description : The fastest route to true HTML/CSS mastery! Need to build a web site? Or update one? Or just create some effective new web content? Maybe you just n
eed to update your skills, do the job better. Welcom...

  #2  HTML: The Complete Reference
       Auteur      : ['Thomas Powell']
       Categories  : None
       Similarite  : 61.7%
       Description : The Definitive Guide to HTML and CSS--Fully Updated Written by a Web development expert, the fifth edition of this trusted resource has been thoroug
hly revised and reorganized to address HTML5, the re...
```

Figure 4 : 2 exemples de recommandations par similarité

Approche par GNN

Pour cette seconde approche, le principe est une recommandation collaborative des livres. Le but est de recommander un livre qu'un utilisateur A a aimé à un utilisateur B qui aurait des goûts similaires. Cela est possible grâce au *Book Crossing Dataset* qui comprend les notes des utilisateurs.

Nous avons vu plus haut le prétraitement qui a été effectué sur le dataset.

Cette approche par GNN est inspirée d'un projet de système de recommandation avec des Graph Neural Networks (GNN) pour le cours Stanford CS224W (voir Sources)

Séparation train/test au niveau des arêtes

Après le prétraitement du dataset, un split 80/20 a été appliqué sur les arêtes afin de constituer le train et test set.

Cependant, la librairie PyTorch Geometric, utilisée pour la construction du modèle GNN, n'accepte que des indices continus pour les nœuds du graphe. Il a donc été nécessaire de réencoder les identifiants des utilisateurs et des livres afin de les mapper sur des intervalles continus $[0, N[$.

De plus, les utilisateurs et livres absents du train set ont dû être supprimés. En effet, le modèle ne peut pas générer de recommandations pour un utilisateur ou un livre jamais observé lors de l'entraînement, puisqu'aucune représentation (embedding) ne peut être apprise pour ces nœuds.

Construction du graphe biparti utilisateur-livre

Ensuite, il a fallu construire le graphe biparti utilisateur–livre. Ce type de graphe comporte deux catégories distinctes de nœuds, ici : des nœuds *utilisateurs* et des nœuds *livres*.

Dans un graphe biparti, les arêtes ne peuvent relier que des nœuds appartenant à des classes différentes : un utilisateur peut être connecté à un livre, mais jamais à un autre utilisateur, et inversement.

Le graphe est non orienté, ce qui signifie que chaque interaction est représentée dans les deux sens. Concrètement, une arête entre l'utilisateur *u3* et le livre *l1* apparaît sous la forme : *u3* → *l1* et *l1* → *u3*.

Cette duplication est nécessaire pour PyTorch Geometric, qui représente les arêtes sous forme de paires directionnelles, même lorsque le graphe est non orienté.

Modèle LightGCN

Pour le modèle, nous avons utilisé *LightGCN*, comme dans le projet du cours de Stanford mentionné précédemment. Pour les graphes utilisateur–item, *LightGCN* est reconnu pour être plus simple, plus rapide et souvent plus performant que les GCN classiques. Sa particularité est de supprimer toutes les transformations linéaires et les fonctions d'activation, afin de ne conserver que l'opération essentielle à la recommandation : l'agrégation des voisins.

Points clés

- La couche LGConv applique une agrégation normalisée des voisins, sans poids ni ReLU.
- LightGCN utilise une seule matrice d'embeddings pour tous les nœuds.
- Chaque nœud reçoit un embedding initial aléatoire.

Entraînement

L'entraînement du modèle repose sur un dataloader qui, à chaque itération, sélectionne un lot d'utilisateurs. Pour chacun d'eux, un livre avec lequel une interaction existe est choisi comme exemple positif, tandis qu'un livre jamais lu est tiré au hasard comme exemple négatif. Ces triplets constituent la base de l'apprentissage BPR, qui vise à apprendre un ordre de préférence entre des livres positifs et négatifs.

Durant la propagation avant, chaque couche met à jour les embeddings en s'appuyant sur la structure du graphe, et l'embedding final d'un nœud est obtenu en moyennant les embeddings produits à chaque couche.

La rétropropagation utilise la somme de la perte Bayesian Personalized Ranking (BPR) et d'une régularisation L2. La perte BPR compare le score attribué à un livre positif et à un livre

négatif pour un même utilisateur, et encourage le modèle à attribuer un score plus élevé aux interactions positives. La régularisation L2, appliquée aux embeddings initiaux, limite leur amplitude et réduit le risque de surapprentissage. Ensemble, ces deux composantes assurent un apprentissage stable et cohérent aumodèle LightGCN.

Évaluation

L'évaluation consiste d'abord à calculer un score pour toutes les paires utilisateur–livre, en excluant les interactions présentes dans l'ensemble d'entraînement afin d'éviter de recommander des livres déjà lus. Pour chaque utilisateur, les livres sont ensuite triés selon leur score, et les K meilleurs sont retenus comme recommandations. Ces prédictions sont comparées aux interactions réelles du jeu de test, ce qui permet de mesurer la performance du modèle.

Les métriques utilisées pour évaluer le modèle sont le Recall@K et la Precision@K. Le Recall@K mesure la proportion de livres correctement prédits parmi l'ensemble des livres réellement lus par l'utilisateur dans le jeu de test. La Precision@K mesure quant à elle la proportion de livres correctement recommandés parmi les K livres proposés. Ensemble, ces deux métriques permettent d'évaluer la pertinence et l'utilité des recommandations générées par le modèle.

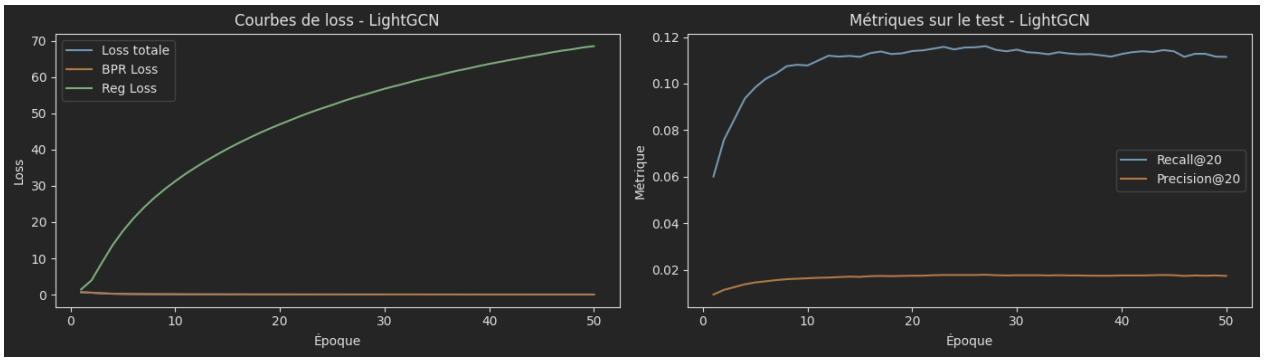

Figure 5 : Evaluation du modèle LightGCN

Les meilleurs scores obtenus sont un Recall@20 de 0.1161 et une Precision@20 de 0.0179. Cela signifie que le modèle retrouve environ 11,6 % des livres réellement lus dans le test set, mais seulement 1,79 % des recommandations du Top-20 sont correctes. Le modèle identifie donc certains livres pertinents, mais le Top-20 contient encore beaucoup de bruit.

La BPR Loss diminue, ce qui montre que l'apprentissage progresse, tandis que l'augmentation de la régularisation L2 suggère qu'un ajustement des hyperparamètres serait nécessaire pour améliorer la stabilité et la qualité des recommandations.

Génération de recommandations

```
Utilisateur exemple : 268032

--- Livres lus dans le TRAIN (connus du modèle) ---
Livres lus (train) — User 268032 : 24 livres
```

24 rows 24 rows × 3 cols

	ISBN	Book-Title	Book-Author
45	0671888587	I'll Be Seeing You	Mary Higgins Clark
408	0316666343	The Lovely Bones: A Novel	Alice Sebold
914	0345391802	The Hitchhiker's Guide to the Galaxy	Douglas Adams
953	0440211727	A Time to Kill	JOHN GRISHAM
1025	0449134482	Dances With Wolves	Michael Blake
1105	0060928336	Divine Secrets of the Ya-Ya Sisterhood: A Novel	Rebecca Wells
1209	0380018179	The Thorn Birds	Colleen McCullough
1322	0451169530	The Stand: Complete and Uncut	Stephen King
2445	0440214041	The Pelican Brief	John Grisham
4005	0449221504	K Is for Killer (Kinsey Millhone Mysteries (Pa...	Sue Grafton
4304	0446608262	The Lion's Game	Nelson DeMille
4496	0312274920	Round Ireland With a Fridge	Tony Hawks

Figure 6 : Livres lus dans le train set (connus du modèle)

```
--- Livres lus dans le TEST (inconnus du modèle) ---
Livres lus (test) — User 268032 : 9 livres
```

9 rows 9 rows × 3 cols

	ISBN	Book-Title	Book-Author
27	0345402871	Airframe	Michael Crichton
621	0553576925	Hiding in the Shadows (Shadows Trilogy (Paperb...	Kay Hooper
888	0440218535	Smilla's Sense of Snow	Peter Hoeg
2441	0553279912	A Is for Alibi (Kinsey Millhone Mysteries (Pap...	SUE GRAFTON
4479	0312924585	Silence of the Lambs	Thomas Harris
9691	0553580221	False Memory	Dean R. Koontz
10760	0449204324	Blue Highways a Journey Into America	William Least Heat-Moon
15514	014100018X	Chocolat	Joanne Harris
30393	0380729342	Tales from Watership Down	Richard Adams

Figure 7 : Livres lus dans le test set (inconnus au modèle)

```
--- Recommandations LightGCN ---
User 268032 — Livres dans le test : 9 | Hits : 1/10 | Precision@10 : 0.10 | Recall@10 : 0.11
```

10 rows 10 rows × 4 cols

	ISBN	Book-Title	Book-Author	hit
1	0345370775	Jurassic Park	Michael Crichton	False
2	0553279912	A Is for Alibi (Kinsey Millhone Mysteries (Pap...	SUE GRAFTON	True
3	044021145X	The Firm	John Grisham	False
4	0553572997	The Alienist	Caleb Carr	False
5	0446610038	1st to Die: A Novel	James Patterson	False
6	0446359866	Presumed Innocent	Scott Turow	False
7	0345378490	Congo	Michael Crichton	False
8	0553268880	The Prince of Tides	Pat Conroy	False
9	0345369947	The Mummy or Ramses the Damned	Anne Rice	False
10	0553279556	E Is for Evidence: A Kinsey Millhone Mystery (...	Sue Grafton	False

Figure 8 : Livres recommandés

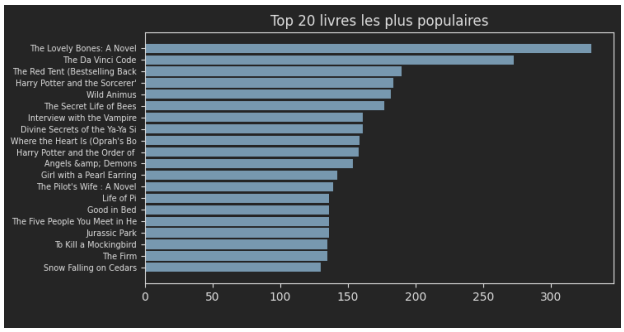

Figure 9 : Top 20 des livres les plus populaires

Lors de la génération des recommandations pour un utilisateur choisi au hasard, un seul des dix livres proposés correspondait réellement à un livre présent dans le jeu de test. À première vue, cela peut laisser penser que le modèle prédit mal, mais une analyse plus fine montre que les recommandations ne sont pas dénuées de pertinence.

Plusieurs livres recommandés appartenaient en effet au **Top 20** des livres les plus populaires, comme *Jurassic Park* et *The Firm*, ce qui révèle un biais de popularité dans les prédictions. Le modèle a également suggéré des ouvrages d'auteurs appréciés par l'utilisateur, notamment Michael Crichton, John Grisham et Sue Grafton , ce qui indique qu'il parvient à capturer certaines préférences générales.

Enfin, plusieurs livres de Sue Grafton ont été lus et recommandés. Cette autrice est connue pour sa série de romans policiers dont les titres suivent l'ordre alphabétique (*A is for…*, *B is for…*, *C is for…*, etc.). Cela suggère que le modèle a identifié que l'utilisateur appréciait cette saga et a appris à la recommander.

Approche 2 : Méthode hybride

Mise en place

La seconde approche repose sur une fusion du filtrage collaboratif et du filtrages sur base du contenu. Les descriptions de livres sont d'abord encodées avec SBERT, produisant un vecteur sémantique de 384 dimensions. Ce vecteur est ensuite projeté en 64 dimensions via une couche linéaire, dont les poids sont appris pendant l'entraînement. Ces embeddings servent d'initialisation pour les nœuds « livre » dans LightGCN, ce qui permet à deux ouvrages textuellement proches d'être déjà représentés de manière similaire avant la première propagation.

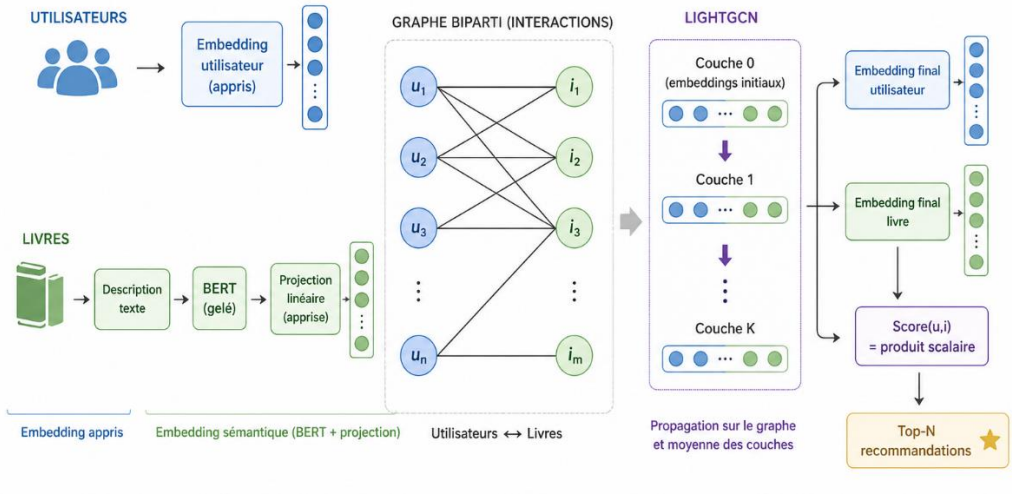

Figure 10 – Pipeline de HybridLightGCN

Pipeline de HybridLightGCN

L'architecture HybridLightGCN reste identique au LightGCN classique : même graphe biparti utilisateur-livre, mêmes couches LGConv, même agrégation moyenne, même BPR loss et même score pour la recommandation. La seule différence concerne l'initialisation des nœuds. Les utilisateurs conservent une initialisation aléatoire et leurs embeddings sont appris uniquement à partir des interactions. Les livres, eux, bénéficient d'une initialisation guidée par leur contenu textuel, que LightGCN affine ensuite grâce au filtrage collaboratif.

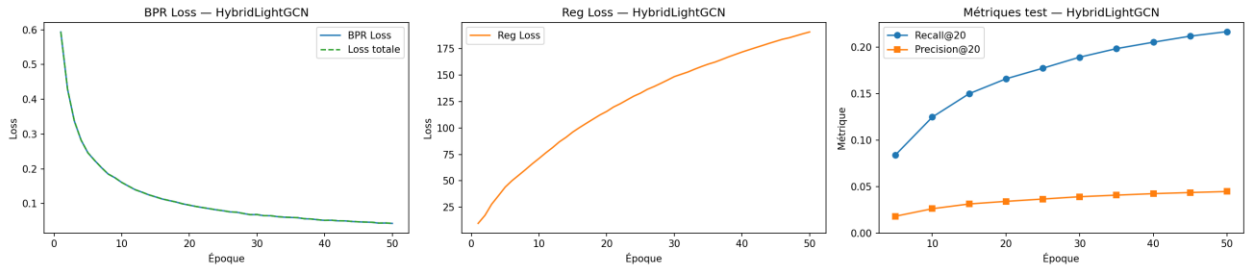

Figure 11 - Courbes d'apprentissage

Les courbes montrent un apprentissage stable du modèle HybridLightGCN. La BPR Loss diminue rapidement au fil des époques, ce qui indique une bonne convergence et une amélioration progressive de la pertinence des recommandations.

Les métriques de Recall@20 et Precision@20 augmentent régulièrement durant l'entraînement. Cela suggère que le modèle apprend des relations pertinentes entre utilisateurs et items.

Cependant, les performances finales restent décevantes car le modèle est encore sous-entraîné. En effet, différentes optimisations ont été mises en place afin de réduire le temps d'exécution de plusieurs heures à quelques minutes.

Les résultats obtenus restent néanmoins cohérents et montrent qu'un entraînement plus long ainsi qu'un ajustement des hyperparamètres pourraient permettre d'améliorer significativement les performances.

Analyse des résultats

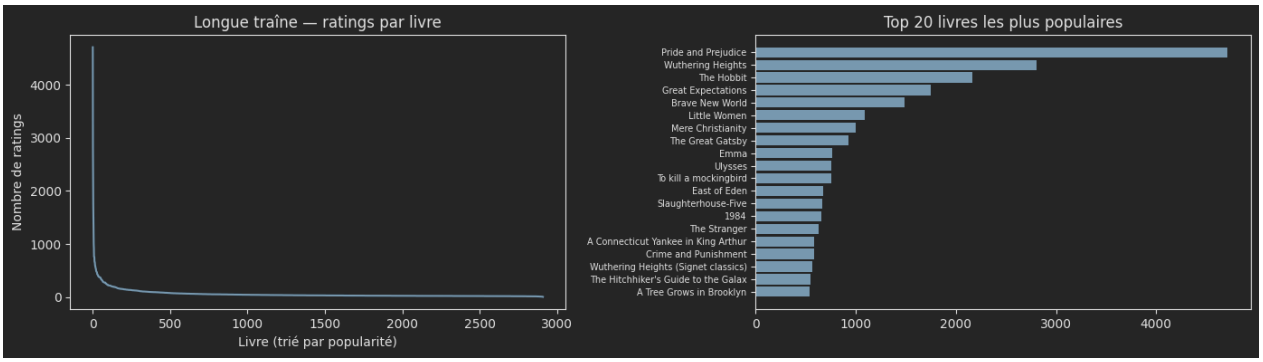

Figure 12 : Répartition du dataset

[BERT+KNN]

rang	titre	score	hit
1	Perelandra: A Novel	1.000	✅
2	Touching the Void: The True Story of One Man's Miraculous Survival	0.630	❌
3	Enduring Love	0.626	❌
4	Typee: Peep at Polynesian Life	0.625	❌
5	Typee: A Peep at Polynesian Life	0.625	❌
6	Carry Me Across the Water	0.621	❌
7	NEW EARTH	0.619	❌
8	The immoralist	0.615	❌
9	The Immoralist	0.615	❌
10	Stranger in a Strange Land : (Science Fiction Book Club Ltd Edition)	0.614	❌

Figure 13 : Résultat de la version BERT + KNN

```
[ GNN hybride ]
                                    titre  score hit
rang
1     The Hitchhiker's Guide to the Galaxy  0.933  ❌
2                Stranger in a Strange Land  0.872  ❌
3                    The War of the Worlds  0.869  ❌
4                      Perelandra: A Novel  0.868  ✅
5                            Lord of Light  0.863  ❌
6                   The Martian Chronicles  0.861  ❌
7                    That Hideous Strength  0.860  ❌
8                                 The Hobbit  0.849  ❌
9                 The Left Hand of Darkness  0.846  ❌
10                      The Battle for God  0.846  ❌
```

Figure 14 : Résultat de la version hybride

Les résultats montrent que l'on fait face à un biais de popularité, avec notamment Le Hobbit proposé alors qu'il n'a pas forcément de rapport avec les autres. Ce biais est expliqué car 34.2% des ratings du dataset sont concentrées sur les 5% des livres les plus populaires. Ils sont donc plus vus et le modèle s'entraîne plus dessus.

Cependant on peut aussi se rendre compte que les scores du GNN hybrides sont bien meilleurs que ceux de la méthode BERT+KNN, ce qui confirme qu'une fusion était intéressante, puisque le système de recommandations propose des livres plus cohérents. Ce qui est logique puisque la méthode ne se base pas seulement sur le fait de trouver le plus proche voisin, mais également sur les notes d'autres utilisateurs, ce qui permet de trouver des profils similaires.

Conclusion

Ce projet nous a permis d'explorer plusieurs approches de recommandation et de comprendre leurs mécanismes respectifs et tout en essayant de trouver une approche hybride plutôt que deux modèles distincts. Au-delà de l'implémentation, il nous a surtout amenés à réfléchir à la complémentarité entre le filtrage collaboratif et filtrage basé sur le contenu, et à concevoir une architecture capable de tirer le meilleur des deux.

Les principales difficultés rencontrées proviennent du dataset. Il est en effet complexe de trouver un jeu de données à la fois riche en résumés, suffisamment dense en interactions utilisateur-livre et correctement distribué sur l'ensemble des ouvrages. Malgré ces contraintes, nos modèles obtiennent des performances cohérentes et dépassent nettement une recommandation aléatoire ce qui était notre objectif minimal.

Nous observons cependant deux limites classiques des systèmes de recommandation.

- Le biais de popularité : les livres les plus lus ou les mieux notés dominent les recommandations, ce qui réduit la diversité et invisibilise les ouvrages moins populaires mais potentiellement pertinents.
- Le problème du cold start : les nouveaux livres ou les nouveaux utilisateurs disposent de peu ou pas d'interactions, ce qui rend difficile l'apprentissage de représentations fiables.

Plusieurs pistes d'amélioration peuvent être envisagées :

- Corriger les doublons dans le dataset afin d'éviter que certains livres apparaissent plusieurs fois sous des identifiants différents et biaisent les recommandations
- Réduire le biais de popularité via un rééchantillonnage des interactions ou une pondération pénalisant les livres trop fréquents.
- Enrichir le filtrage basé sur le contenu et mieux gérer le cold start, en intégrant d'autres sources textuelles (auteurs, métadonnées, genres, mots-clés)
- Utiliser un dataset plus équilibré ou appliquer des techniques de data augmentation pour compenser la faible densité d'interactions.
- Corriger les doublons dans le dataset afin d'éviter que certains livres apparaissent plusieurs fois sous des identifiants différents et biaisent les recommandations
- Explorer d'autres modèles comme des mécanismes d'attention entre contenu et interactions ou encore un auto-encoder.

Sources :

- https://colab.research.google.com/drive/1VQTBxJuty7aLMepjEYE-d7E9kjo51CA1?usp=sharing#scrollTo=6RNPSfjWjV25
- https://pytorch-geometric.readthedocs.io/en/latest/

- *GoodReads 100K* : https://www.kaggle.com/datasets/mdhamani/goodreads-books-100k
- *Book Crossing Dataset:* https://www.kaggle.com/datasets/syedjaferk/book-crossing-dataset?select=BX-Books.csv
- *Amazon Book Reviews:* https://www.kaggle.com/datasets/mohamedbakhet/amazon-books-reviews